INTRODUCTION TO

AGENTIC AI

Building Intelligent Orchestrated Systems in the Cloud

What is Agentic AI?

Agentic AI systems are autonomous agents that can perceive their environment, **make decisions**, and **take actions** to achieve specific goals without constant human intervention.

Key Characteristics

Autonomy

Operates independently with minimal supervision

Perception

Senses and interprets environmental data

Decision-Making

Evaluates options and chooses actions

Goal-Oriented

Works towards defined objectives

Benefits of Agentic AI

1. **Increased Efficiency**
   Automates complex tasks and workflows

2. **24/7 Availability**
   Operates continuously without breaks

3. **Scalability**
   Handles growing workloads seamlessly

4. **Consistency**
   Reduces human error and variability

5. **Data-Driven Insights**
   Analyzes patterns humans might miss

6. **Cost Reduction**
   Optimizes resource allocation

Types of Agentic AI

Voice Agents

Natural language processing systems
Speech recognition & synthesis
Conversational AI interfaces

Examples: Alexa, Siri, Google Assistant

Vision Agents

Computer vision & image recognition
Object detection and tracking
Visual scene understanding

Examples: Autonomous vehicles, Security systems

Types of Agentic AI (Continued)

Physical Agents

Robotic systems with sensors & actuators
Physical world interaction
Manufacturing & logistics automation

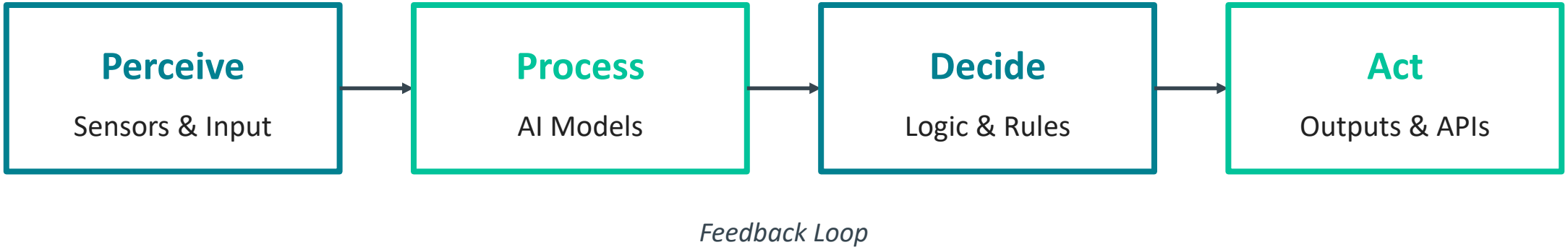
Examples: Industrial robots, Drones, Warehouse automation

Software Agents

Digital assistants & chatbots
Workflow automation systems
Code generation & development tools

Examples: GitHub Copilot, ChatGPT, RPA bots

How Agentic AI Works



Core Components

- Perception Layer: Data ingestion from sensors, APIs, or user input
- Processing Layer: Machine learning models, NLP engines, computer vision
- Decision Layer: Rule engines, optimization algorithms, planning systems
- Action Layer: API calls, robotic controls, user interface responses

How Agents Actually Work

It's More Than Just a Prompt!

1. **Prompt Analysis**
   Agent analyzes user intent, extracts entities, identifies task type

2. **Tool/Service Discovery**
   Agent has a 'toolbox' of available services (weather API, calculator, database, etc.)

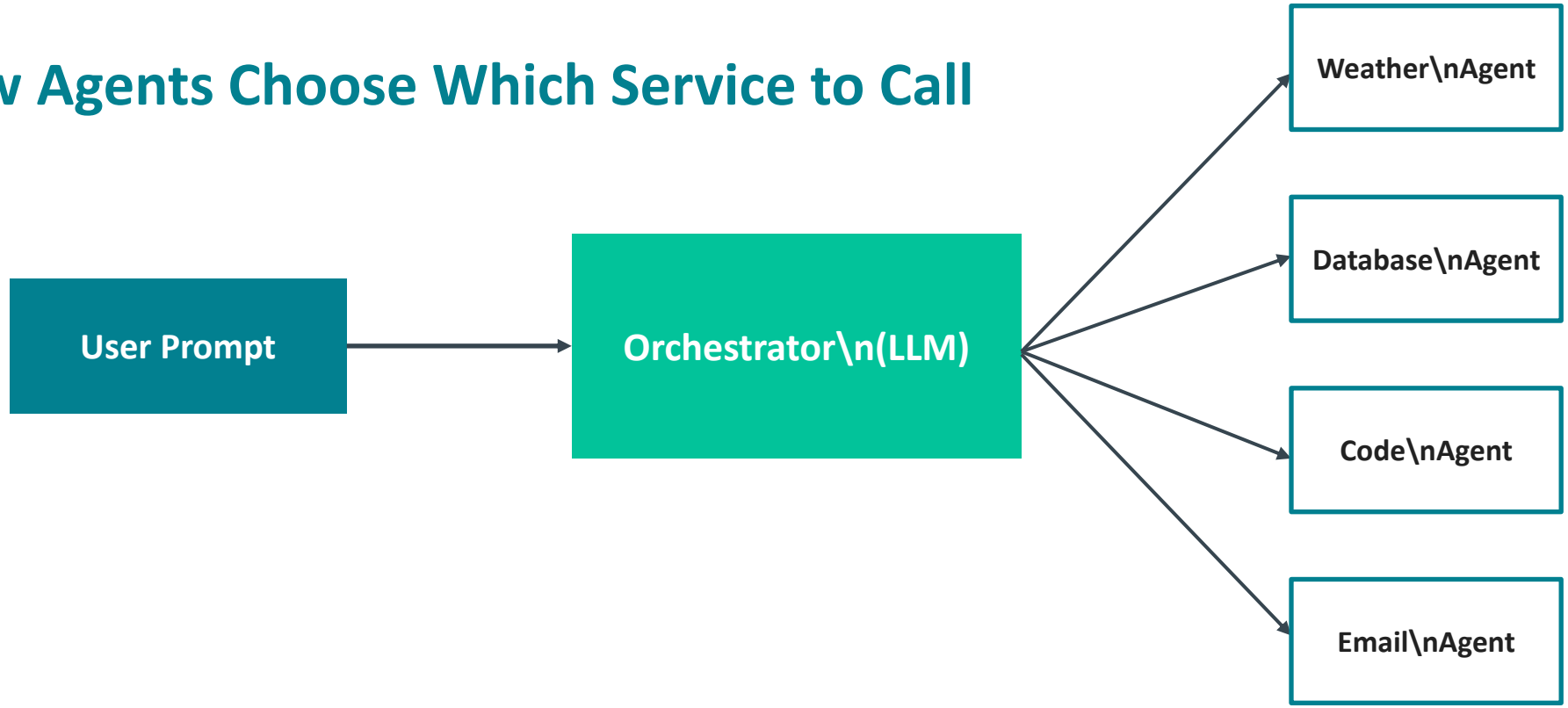
3. **Decision Logic**
   LLM decides which tool(s) to call based on function descriptions and context

4. **Tool Execution**
   Agent calls the selected API/service with extracted parameters

5. **Result Synthesis**
   Agent combines tool results with LLM reasoning to generate final response

Agent Orchestration: The Router

How Agents Choose Which Service to Call



Decision Criteria:

- Function descriptions tell the LLM what each agent can do
- LLM matches user intent to the best-suited agent(s)
- Example: "What's the weather?" → Weather Agent | "Send email to John" → Email Agent

Function Calling in Action

OpenAI Function Calling

```
# Define available tools/functions
tools = [
    {
        "type": "function",
        "function": {
            "name": "get_weather",
            "description": "Get current weather for a location",
            "parameters": {
                "type": "object",
                "properties": {
                    "location": {"type": "string", "description": "City
name"}
                }
            }
        }
    },
    {
        "type": "function",
        "function": {
            "name": "send_email",
            "description": "Send an email to a recipient",
            "parameters": {
                "type": "object",
                "properties": {
                    "to": {"type": "string"},
                    "subject": {"type": "string"},
                    "body": {"type": "string"}
                }
            }
        }
    }
]
```

```
# Agent call with tools
response = client.chat.completions.create(
    model="gpt-4",
    messages=[{"role": "user", "content": "What's the weather in Paris?"}],
    tools=tools,  # LLM chooses which function to call
    tool_choice="auto"
)

# LLM decides: get_weather(location="Paris")
```

Agent Orchestration Patterns

Common Architectures for Multi-Agent Systems

Sequential Chain	Output of Agent A → Input of Agent B → Agent C *Data processing pipelines, multi-step analysis*	Use: Data processing pipelines
Parallel Execution	Multiple agents work simultaneously, results merged *Fact-checking from multiple sources, A/B testing*	Use: Fact-checking from multiple sources
Router Pattern	Central orchestrator routes to specialized agents *Customer support (billing vs technical vs sales)*	Use: Customer support (billing vs technical vs sales)
Hierarchical	Manager agent coordinates worker agents *Complex projects (research → write → edit → publish)*	Use: Complex projects (research → write → edit → publish)
Collaborative	Agents negotiate and debate to reach consensus *Decision-making systems, multi-perspective analysis*	Use: Decision-making systems

Multi-Agent Orchestration Code

LangGraph - Agent Workflow Framework

```
from langgraph.graph import StateGraph, END
from langchain_openai import ChatOpenAI

# Define agent state
class AgentState(TypedDict):
    user_query: str
    agent_chosen: str
    result: str

# Initialize agents
weather_agent = ChatOpenAI(model="gpt-4")
email_agent = ChatOpenAI(model="gpt-4")

# Router function - decides which agent to call
def route_query(state: AgentState):
    query = state["user_query"]
    # Use LLM to classify intent
    if "weather" in query.lower():
        return "weather_agent"
    elif "email" in query.lower():
        return "email_agent"
    return "general"

# Build workflow graph
workflow = StateGraph(AgentState)
workflow.add_node("router", route_query)
workflow.add_node("weather_agent", call_weather_agent)
workflow.add_node("email_agent", call_email_agent)

# Define edges (routing logic)
workflow.add_conditional_edges(
    "router",
    lambda x: x["agent_chosen"],
    {"weather_agent": "weather_agent", "email_agent": "email_agent"}
)
workflow.set_entry_point("router")
```

Model Context Protocol (MCP)

Standardized Way for LLMs to Connect to Data & Tools

MCP is an open protocol by Anthropic that lets AI assistants connect to external data sources and tools. Instead of building custom integrations for each tool, **MCP provides a universal interface** - like USB for AI agents.

Standardization
One protocol for all integrations

Bidirectional
Tools can push data to LLMs

Secure
Built-in authentication & permissions

Discoverable
Tools self-describe their capabilities

MCP Architecture

How MCP Connects LLMs to Tools

Your Application (LLM Host)
Claude, ChatGPT, or custom app

↓

MCP Client
Manages connections to MCP servers

↓

MCP Server(s)
Database | Filesystem | API | Slack...

↓

External Resources
Postgres, Files, APIs, Services

MCP Server Implementation

Creating an MCP Server (Python)

```python
from mcp.server import Server
from mcp.types import Tool, TextContent

# Create MCP server
server = Server("weather-server")

# Define tool
@server.list_tools()
async def list_tools() -> list[Tool]:
    return [
        Tool(
            name="get_weather",
            description="Get current weather for a city",
            inputSchema={
                "type": "object",
                "properties": {
                    "city": {"type": "string", "description": "City name"}
                },
                "required": ["city"]
            }
        )
    ]
```

```python
# Implement tool
@server.call_tool()
async def call_tool(name: str, arguments: dict):
    if name == "get_weather":
        city = arguments["city"]
        # Call actual weather API
        weather_data = await fetch_weather(city)
        return [TextContent(
            type="text",
            text=f"Weather in {city}: {weather_data['temp']}°C"
        )]

# Run server
server.run()
```

AI Fundamentals: Key Concepts

Understanding Core AI Parameters

Tokens

Units of text processed by AI models (≈4 chars per token)

```
Input + Output tokens = Total cost
```

Temperature

Controls randomness: 0 = deterministic, 1 = creative

```
0.2 for code, 0.7 for writing, 0.9 for brainstorming
```

Max Tokens

Maximum response length limit

```
GPT-4: 8K-128K tokens, Claude: 200K tokens
```

Rate Limits

API request restrictions (requests per min/day)

```
Tier 1: 500 RPM, Tier 5: 10,000 RPM
```

Azure AI Foundry

Enterprise AI Platform & Agent Builder

Custom Knowledge Bases

Upload documents, PDFs, and data to create domain-specific AI

Agent Builder

Visual interface to create multi-agent workflows with Prompt Flow

Model Fine-Tuning

Customize GPT-4, Llama, Phi models with your training data

Safety & Content Filters

Built-in content moderation and responsible AI guardrails

Vector Search Integration

Connect to Azure Cognitive Search for RAG patterns

Deployment & Scaling

One-click deployment with auto-scaling and load balancing

Access: ai.azure.com | Integrates with Azure ML, OpenAI Service, Cognitive Services

API Integration with Parameters

OpenAI API with Temperature & Token Control

```
from openai import OpenAI

# Initialize client
client = OpenAI(api_key="sk-your-key")

# API call with parameter control
response = client.chat.completions.create(
    model="gpt-4-turbo",
    messages=[
        {"role": "system", "content": "You are a helpful assistant."},
        {"role": "user", "content": "Analyze this data"}
    ],
    temperature=0.7,      # Creativity: 0=focused, 1=creative
    max_tokens=1000,      # Response length limit
    top_p=0.9,            # Nucleus sampling
    frequency_penalty=0,  # Reduce repetition
    presence_penalty=0    # Encourage new topics
)

# Token usage tracking
print(f"Tokens used: {response.usage.total_tokens}")
print(f"Prompt: {response.usage.prompt_tokens}")
print(f"Completion: {response.usage.completion_tokens}")

result = response.choices[0].message.content
```


```
# Token usage tracking
print(f"Tokens used: {response.usage.total_tokens}")
print(f"Prompt: {response.usage.prompt_tokens}")
print(f"Completion: {response.usage.completion_tokens}")

result = response.choices[0].message.content
```

Custom Knowledge: RAG Pattern

Retrieval-Augmented Generation

```
from openai import OpenAI
from pinecone import Pinecone

client = OpenAI(api_key="your-key")
pc = Pinecone(api_key="pinecone-key")
index = pc.Index("knowledge-base")

# User query
query = "What are our AI safety policies?"

# Get embeddings
embedding = client.embeddings.create(
    model="text-embedding-ada-002",
    input=query
).data[0].embedding

# Search vector DB
results = index.query(vector=embedding, top_k=3)

# Build context
context = "\n".join([match.metadata["text"] for match in results.matches])
```

```
# Generate response with context
response = client.chat.completions.create(
    model="gpt-4",
    messages=[
        {"role": "system", "content": f"Context: {context}"},
        {"role": "user", "content": query}
    ]
)
```

Voice AI Integration

Azure Speech Services - Real-time Recognition

```
import azure.cognitiveservices.speech as speechsdk

# Configure Speech Service
speech_config = speechsdk.SpeechConfig(
    subscription="YOUR_KEY",
    region="eastus"
)
speech_config.speech_recognition_language = "en-US"

# Setup audio input
audio_config = speechsdk.audio.AudioConfig(
    use_default_microphone=True
)

# Create recognizer
speech_recognizer = speechsdk.SpeechRecognizer(
    speech_config=speech_config,
    audio_config=audio_config
)

# Start recognition
print("Speak into your microphone...")
result = speech_recognizer.recognize_once_async().get()

# Process result
if result.reason == speechsdk.ResultReason.RecognizedSpeech:
    print(f"Recognized: {result.text}")
    # Send to AI agent for processing
    agent_response = process_command(result.text)
elif result.reason == speechsdk.ResultReason.NoMatch:
    print("No speech could be recognized")
else:
    print(f"Error: {result.reason}")
```

Vision AI Integration

Google Cloud Vision - Object Detection

```
from google.cloud import vision
import io

# Initialize Vision API client
client = vision.ImageAnnotatorClient()

# Load image
with io.open('image.jpg', 'rb') as image_file:
    content = image_file.read()

image = vision.Image(content=content)

# Detect objects
response = client.object_localization(image=image)
objects = response.localized_object_annotations

# Process detected objects
print(f"Number of objects found: {len(objects)}")
for object_ in objects:
    print(f"Object: {object_.name}")
    print(f"Confidence: {object_.score:.2%}")
    print(f"Bounding box vertices:")
    for vertex in object_.bounding_poly.normalized_vertices:
        print(f"  - ({vertex.x:.3f}, {vertex.y:.3f})")

# Send to decision engine
decisions = make_decisions_from_vision(objects)

# Execute actions
for decision in decisions:
    execute_action(decision)
```

Tutorial: Azure AI Foundry Setup

Building Custom AI with Your Data

1. **Create Azure AI Hub**
   Navigate to ai.azure.com → Create new hub → Select subscription & region

2. **Upload Knowledge Base**
   Add documents (PDFs, Word, CSV) → Auto-chunks and indexes content

3. **Configure Model**
   Select GPT-4, Llama, or Phi → Set temperature, max tokens, system prompt

4. **Deploy & Test**
   Deploy to endpoint → Get API key → Test with Prompt Flow or REST API

Resources: ai.azure.com | docs.microsoft.com/azure/ai-services

Tutorial: AI-Powered IDE

Cursor Setup

1. Install Cursor
2. Start coding with AI assistance
3. Use Cmd+K (Mac) or Ctrl+K (Win) for AI edit
4. Use Cmd+L (Mac) or Ctrl+L (Win) for chat
5. Start typing code for suggestions
6. Use entire codebase for agentic experience

Key Features

- Autocomplete: AI code suggestions as you type (Tab to accept)
- Chat: Ask questions about your codebase with Cmd/Ctrl+L
- Edit: Select code and use Cmd/Ctrl+K to modify with AI
- Composer: Multi-file editing with Cmd/Ctrl+I

Alternative: Cursor (cursor.com), Tabnine, Amazon CodeWhisperer

Developer Resources & API Endpoints

Where to Get AI Capabilities

Provider	AI Capabilities	Documentation URL
OpenAI	GPT-4, Whisper, DALL-E	`platform.openai.com`
Anthropic	Claude API	`anthropic.com/api`
Google Cloud AI	Vision, Speech, Vertex AI	`cloud.google.com/ai`
Azure AI Services	OpenAI, Cognitive, AI Foundry	`azure.microsoft.com/ai`
AWS Bedrock	Claude, Llama, Titan models	`aws.amazon.com/bedrock`
Hugging Face	Open models & inference API	`huggingface.co`
Replicate	Model hosting & deployment	`replicate.com`

Configuration Portal Example

OpenAI Platform Dashboard

OpenAI Platform

API Keys | Usage | Models | Settings

API Keys

`sk-proj-abc...xyz123` **Copy**

Model Configuration

- Model: gpt-4-turbo
- Max Tokens: 4096
- Temperature: 0.7
- Rate Limit: 500 requests/min

Cloud Cost Management

Optimizing AI Agent Costs

Token Optimization	1. Use shorter prompts 2. Implement caching 3. Stream responses	**30-50% reduction**
Model Selection	1. Use GPT-3.5 for simple tasks 2. Reserve GPT-4 for complex reasoning 3. Try open-source alternatives	**60-80% on simple tasks**
Rate Limiting	1. Implement request queues 2. Batch similar requests 3. Use exponential backoff	**Prevents overages**
Monitoring & Alerts	1. Set spending limits 2. Track token usage per feature 3. Alert on anomalies	**Budget control**

THANK YOU

Questions?

Start building intelligent agent systems today!